

Dept.cpp

```
#include "Dept.hpp"
#include<iostream>
#include<tchar.h>

Dept::~Dept()
{
    delete[]scores;
}
Dept::Dept(int size)
{
    this->size =size;
    scores = new int[size];
}
Dept::Dept(const Dept& dept)
{
    this -> size = dept.size;
    scores = new int[size];
    for(int i=0;i<size;i++)
    {
        scores[i]= dept.scores[i];
    }
}
int Dept::getSize()const
{
    return this->size;
}
bool Dept::isOver60(int index) const
{
    if(index<0 || index>=size)
    {
        return false;
    }
    return scores[index]>=60;
}
void Dept::read()
{
    std::wcout<<"Student scores enter: ";
```

```

        for(int i = 0 ;i<size; i++)
        {
            std::cin>>scores[i];
        }
    }
    int Dept::countPass(const Dept& dept)
    {
        int count = 0;
        for(int i =0; i<dept.getSize();i++)
        {
            if(dept.scores[i]>=60)
            {
                count++;
            }
        }
        return count;
    }

```

}

Dept.hpp

```

#ifndef __DEPT_HPP__
#define __DEPT_HPP__

```

```

class Dept
{
    private:
        int size;
        int *scores;
    public:
        Dept(int size);
        Dept(const Dept& dept);
        ~Dept();

        int getSize()const;
        void read();
        bool isOver60(int index) const;
        int countPass(const Dept& dept);
};

```

```

#endif
#include<iostream>

```

```
main.cpp
#include "Dept.cpp"

int main(int argc, char *argv[])
{

    Dept dept(10);
    dept.read();

    int numPassed = dept.countPass(dept);
    std::cout<<"over 60 student count:"<<numPassed<<std::endl;

    return 0;

}
```

요구사항 분석 및 문제 해결 아이디어

요구사항 분석

목적

학생 성적 입력 및 통계 처리 프로그램 작성.

주어진 성적 데이터에서 60 점 이상의 성적을 받은 학생 수를 계산.

클래스 Dept

멤버 변수:

size: 성적 배열의 크기.

`scores`: 학생 성적을 저장하는 동적 배열.

생성자:

`Dept(int size)`: 배열 크기를 설정하고 동적 배열을 초기화.

`Dept(const Dept& dept)`: 깊은 복사를 수행하는 복사 생성자.

소멸자:

`~Dept()`: 동적 배열 메모리 해제.

멤버 함수:

`getSize() const`: 배열의 크기를 반환.

`isOver60(int index) const`: 주어진 인덱스의 성적이 60 점 이상인지 여부를 반환.

`read()`: 학생 성적을 입력받음.

`countPass(const Dept& dept)`: 60 점 이상인 학생 수를 계산.

문제 해결 아이디어

메모리 관리

동적 배열을 사용하기 때문에 메모리 해제를 위해 소멸자를 구현함.

복사 생성자를 통해 깊은 복사를 수행하여 객체의 독립성을 유지.

성적 입력 및 검증

`read()` 함수에서 표준 입력을 통해 학생 성적을 입력받음.

`isOver60(int index)` 함수를 통해 유효한 인덱스 범위를 검증하고, 해당 인덱스의 성적이 60 점 이상인지 확인.

성적 통계 계산

`countPass(const Dept& dept)` 함수에서 전체 성적 배열을 순회하며 60 점 이상인 성적의 개수를 계산.

주어진 배열 크기를 반환하는 `getSize()` 함수를 통해 반복 범위를 설정.

프로그램 실행 흐름

`main` 함수에서 `Dept` 객체를 생성하고, `read()` 함수를 통해 성적을 입력받음.

`countPass()` 함수를 호출하여 60 점 이상인 학생 수를 계산하고 결과를 출력.

이로써 주어진 코드는 학생 성적 입력 및 60 점 이상인 학생 수 계산을 효율적으로 수행할 수 있도록 설계됨.